# OPSC6312

## Portfolio of Evidence

Kylah Irvine - ST10435066

# Table of Contents

Introduction ..... 4
Project Development Journey ..... 5
Initial Planning ..... 5
Authentication System Development ..... 5
Map Integration ..... 5
Post Creation System ..... 5
Data Storage Implementation ..... 6
Map Marker System ..... 6
Settings Menu ..... 6
Application Features ..... 7
User Registration ..... 7
User Login ..... 7
Creating a Bug Post ..... 7
Viewing Saved Posts ..... 7
Viewing Bug Pins ..... 7
Theme Switching ..... 7
Logging Out ..... 7
Language ..... 7
Technical Implementation ..... 8
Programming Language ..... 8
Authentication Technology ..... 8
Map Technology ..... 8
Local Data Storage ..... 8
Logging System ..... 8
Language option ..... 8
Error Handling ..... 9
Try-Catch Implementation ..... 9
Location Permission Handling ..... 9
Testing ..... 10
Authentication Testing ..... 10
Post Testing ..... 10
Map Testing ..... 10

Appendices ..... 12
Appendix A ..... 12
Appendix B ..... 18

# Introduction

The app was initially a more social platform, but has morphed into a more private app for recording where you've seen bugs and when you saw them. It is good for people who like keeping records of these things, like bird watchers but for bugs. It is an android exclusive since that is the platform we were told to make it for.

# Project Development Journey

## Initial Planning

During part 1 I researched decently about how to go about making this application for this POE. I had some decent preexisting apps to look at as reference and this definitely helped out (See attached copy of part 1 docs for reference). The main goal was to make an app that a user can use to record sightings of bugs with images and locations. It began as a social app, where you'd post for others to see the bugs, but I couldn't shake the idea that it can be used to dox someone (see the retired FNAF AR app as an example, no one could play it on stream as that would dox them. The game did not last long) and eventually I pivoted to focusing more on the single user experience. It was incidentally also more plausible to build in my capacity as a college student since I can't pay for the services that I would need to make a live social app. I may end up making this into an more app for myself after this project, as the concept for it is something I would use.

## Authentication System Development

The login and registration screen comes from the code we were given by our lecturer in semester 1, and I have used it for each Kotlin project since. If it ain't broke, don't fix it. It uses Firebase and works with an email and password. I'm somewhat sure that its encrypted by Firebase. The log out works simply, just a small block of code to remove the flag that tells the app that a user by a certain email is currently allowed access to the app and sends them to the log in screen.

## Map Integration

The app uses the OpenStreetMap REST API to work the map functionality. Its free and I think open source so its much better for me to use. The UI isn't the most detailed but it gets the job done. It wasn't hard to integrate, mostly just importing the dependencies. The app needs permission from the device for the location service to work. It uses the phones location tracking to show where you are in the map. It uses the longitude and latitude values, not place names. That is something I intended however, I wanted the locations to be as accurate as possible so you know exactly where the bug is. The idea of having the pins drop off after some time was left off since it became more of a personal bug tracking app than a social one. That is the way of development though, sometimes you start with one idea and end with another.

## Post Creation System

Making the "post" about the bug was a simple enough thing once I stopped trying to make it fancy. As of writing this report the posts do not show with images but that is since I couldn't get it to work in a way I understand. The image selection system is still there, and it works by letting you pick an image from your phones files. The current location of the device is recorded when you click post and stored locally.

## Data Storage Implementation

I tried to use Firestore but it turned into a whole thing and then there was the offline issues so I decided to keep storage local to the users device. It is good for security and would save money on servers if it were going to production. It was also something I could understand. It works similar to the logging system I came up with. Having the logs of what happened in each session of the apps use is incredible for bug hunting, and that information is also kept on the users device, so they can share those files only if they want to.

## Map Marker System

The posts make pins on the map that you can click on to see which bug was found there and its very fun to use actually. The code will read the stored posts and search for coordinates and then place a pin on those coordinates automatically. The map has all the standard zoom and move around functions a map has but the overlay is not changeable. The idea of having pins fade away became sort of pointless as the app progressed since I would like to see all the bugs, maybe later I could add a filter feature to show only specific pins.

## Settings Menu

The menu was a simple slide out one, we made similar ones before for projects. I like the hamburger icon for it. I made the settings simple and just did the ones needed for the project for now.

# Application Features

## User Registration

When you open the application, you should be greeted with a log in screen. Since you don't have an account yet, you should click the register link below the form. Once there, you should enter your email and a password, these will be used as your log in details. Click register and you should be taken back to the log in screen.

## User Login

Once you are registered, fill in your details into the form and click to log in. You should be logged in now.

## Creating a Bug Post

You should click the button that says add new post, here you can click on the white square to add an image and add a bug name. When you're ready, click on the post button.

## Viewing Saved Posts

You should see your posts under the map, currently they are in text form. You can see the image location, but currently you cannot view the image in the app.

## Viewing Bug Pins

If you look at the map you should see pins that you can click on. These pins correspond to your posts. They are often clustered up, but that is due to the use of the exact coordinates, meaning you can have 2 or more pins just a smidge apart.

## Theme Switching

You can switch the theme of the app by going to the settings menu in the top left and toggling the theme switch.

## Logging Out

You should see the button on the main screen that says Log out, when clicked it should log you out.

## Language

I implemented this as a system level system. When you change your device language, the app should also change to that language. It is currently set up to be Afrikaans or English.

# Technical Implementation

## Programming Language

We had to use Kotlin and Android Studio as that's what was taught, but it was certainly not without issues. On the Virtual Machine Android Studio wasn't so bad since it has the RAM to run it, and BlueStacks gave me no issues, but on my personal laptop it was not so smooth. I had to use my laptop to do this project because emulators don't work with testing location functions. Kotlin is also not my favourite language. I doubt I would ever be an Android developer.

## Authentication Technology

For the log in I used the same code we got from Sir in semester 1. Not much changed with it. It is a simple Firebase form that uses emails and passwords to distinguish accounts.

## Map Technology

I used the OpenStreetMap REST API to make the map functionality. I followed the instructions on the website of how to implement it and overall it works nicely.

## Local Data Storage

I stored the post details locally using JSON, this way it works offline and doesn't need storage servers.

## Logging System

For this project I made an entire logging system simply because the regular logcat was too confusing and didn't tell me what I wanted to know. It still uses the standard Log system but it logs it in a document for me to read over. Almost every time the app does something, its logged. Nothing in that app happens without me knowing about it. It wasn't too hard to make using Copilot but getting it to be what I wanted it to be instead of generic AI code took time. Very worth it however, I like that my idea was actually possible and helpful. Logs and regions are my two lifesavers in Kotlin. The logs tell me when something went wrong and what the system said went wrong. This let me debug the app way easier, as looking up error codes is very easy when you have the exact context the error occurred in in plain text. Its not perfect as this was a feature for me and not really for the app itself but I am proud of it. Theres also many many try catches everywhere, which surprisingly doesn't always prevent crashing when playing with the code. But you'll know what made it crash at least.

## Language option

In the strings.xml I was given a prompt by Android studio to add translations, and following its suggestions I learned that it has an automatic language switcher. If you set the locale and translate for it then when you change your system settings to that language, it will change in the app too.

# Error Handling

## Try-Catch Implementation

I added try catches wherever I could since it would help record when something went wrong. Since Kotlin is temperamental with me I definitely needed them. It helps with making sure that the app doesn't crash when there's exceptions like bad redirects or lacking permissions.

## Location Permission Handling

When you launch the app it automatically checks and gets permissions for both using the phones location services and using the map API in general. It does this when the app launches so it happens without the user realizing it. If any of these permissions fail the user gets an error message.

# Testing

I cannot write unit tests to save my life, let alone in Kotlin. Even with the best AI coder I couldn't do it simply because they don't make sense to me, just doesn't compute in my brain. But I can do manual testing, so there's that.

## Authentication Testing

- Tried logging in with account that didn't exist
  - Expected: App doesn't let me in
  - Actual: App doesn't let me in
  - Result: Pass
- Tried registering with an invalid email
  - Expected: Told to use valid email
  - Actual: Told to use valid email
  - Result: Pass
- Tried registering with correct details
  - Expected: Account registered
  - Actual: Account registered
  - Result: Pass
- Tried logging in with existing valid details
  - Expected: User is logged in
  - Actual: User is logged in
  - Result: Pass
- Tried logging out
  - Expected: User is logged out
  - Actual: User is logged out
  - Result: Pass

## Post Testing

- Tried creating post with no bug name
  - Expected: Post created
  - Actual: Post created
  - Result: Pass
- Tried erasing post history
  - Expected: Post history erased
  - Actual: Post history erased
  - Result: Pass

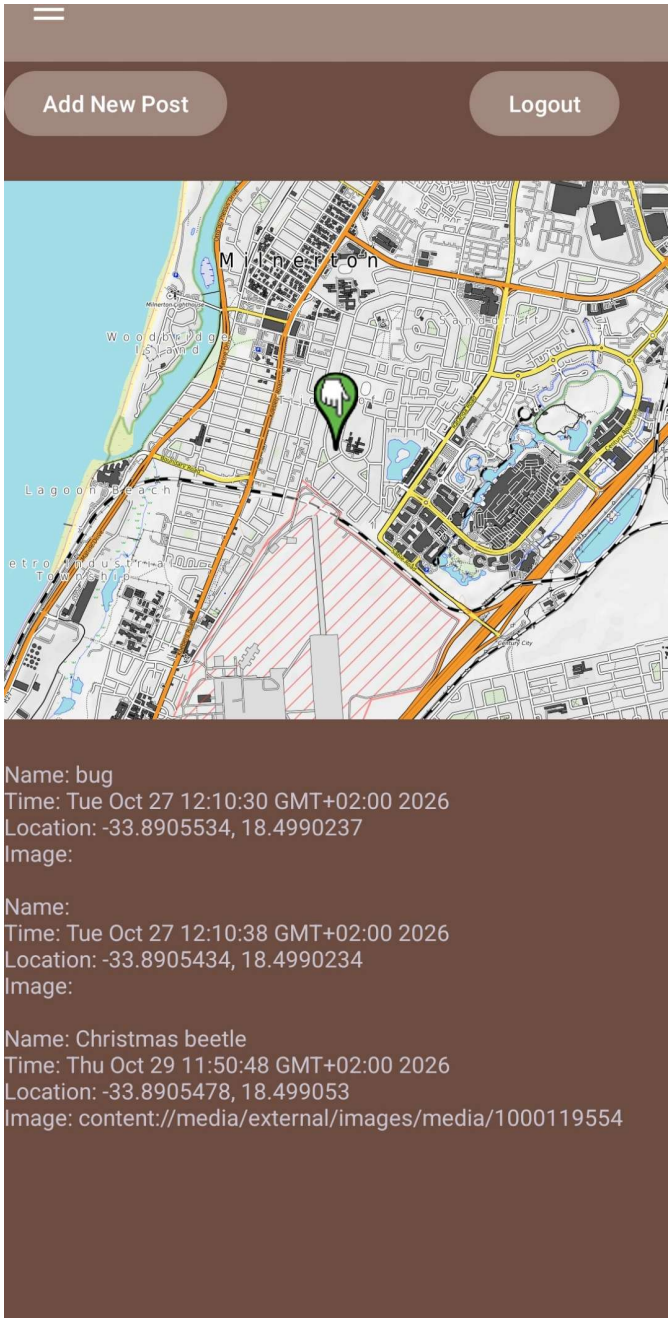
## Map Testing

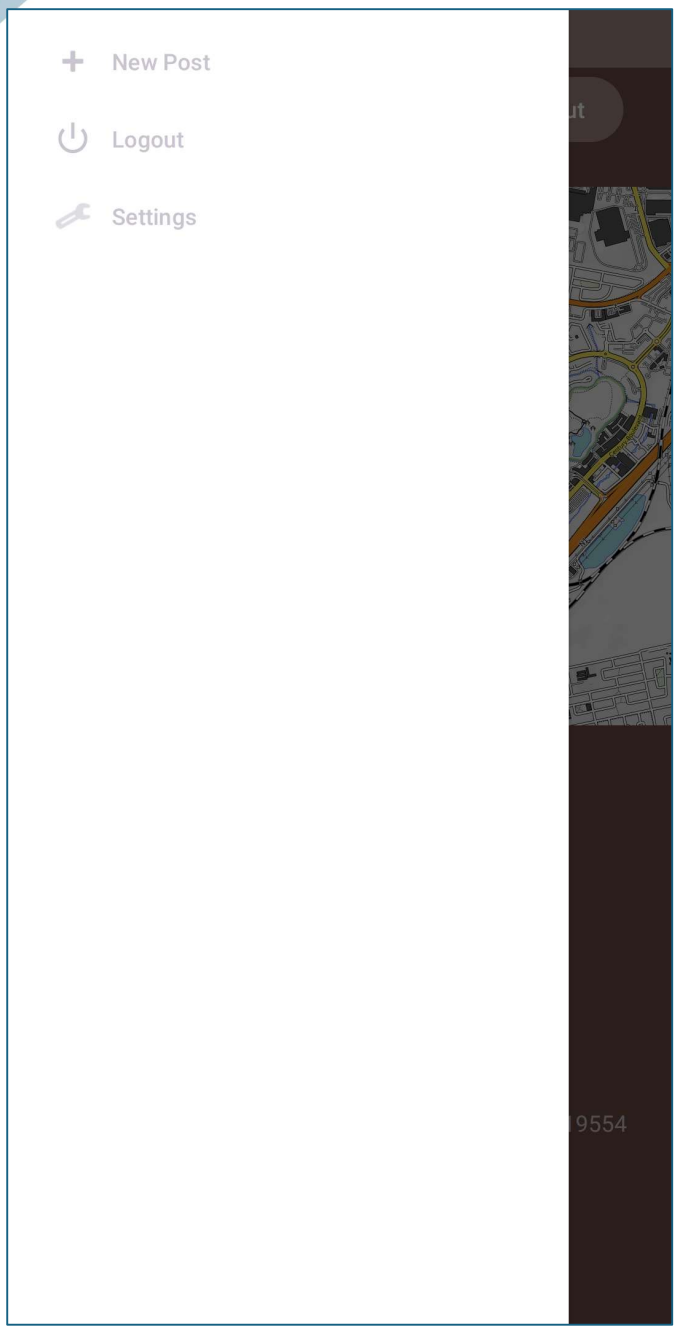
- Tested if map shows when opening the app
  - Expected: It shows and is functional
  - Actual: It shows and is functional
  - Result: Pass

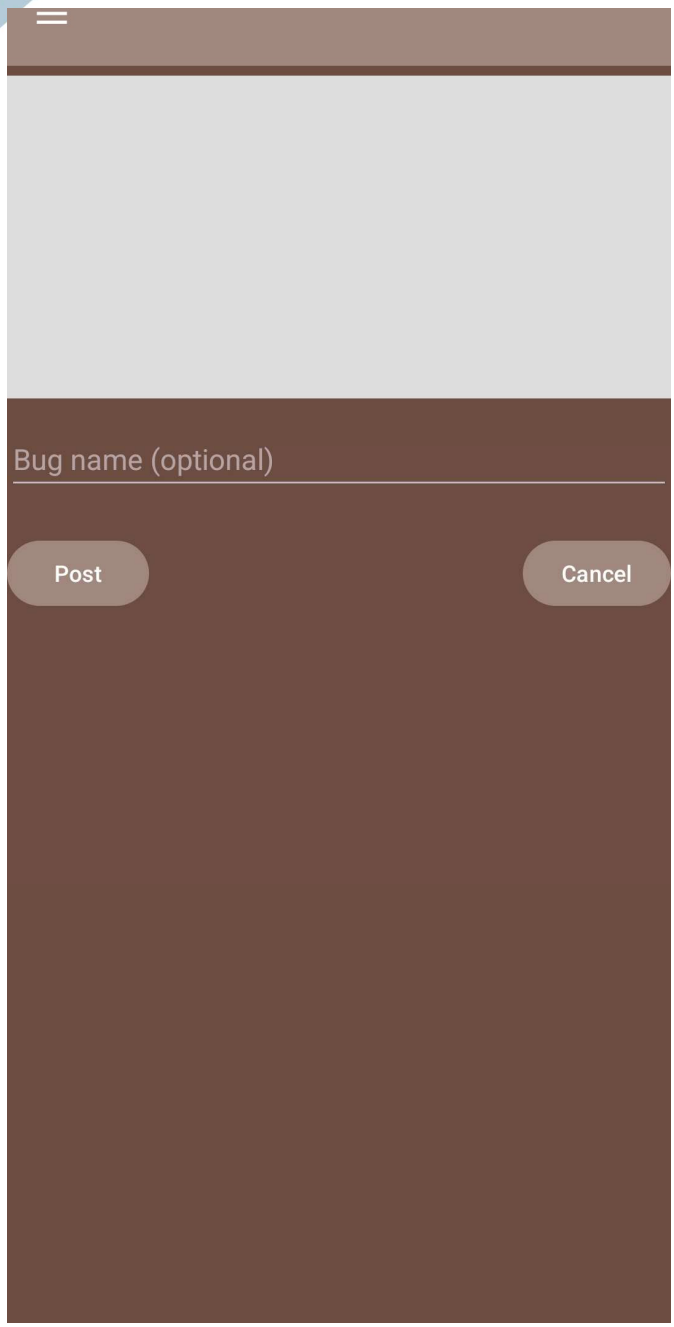
- Tested if pins appear on map
    - Expected: pins appear in correct place on map
    - Actual: pins appear in correct place on map
    - Result: Pass

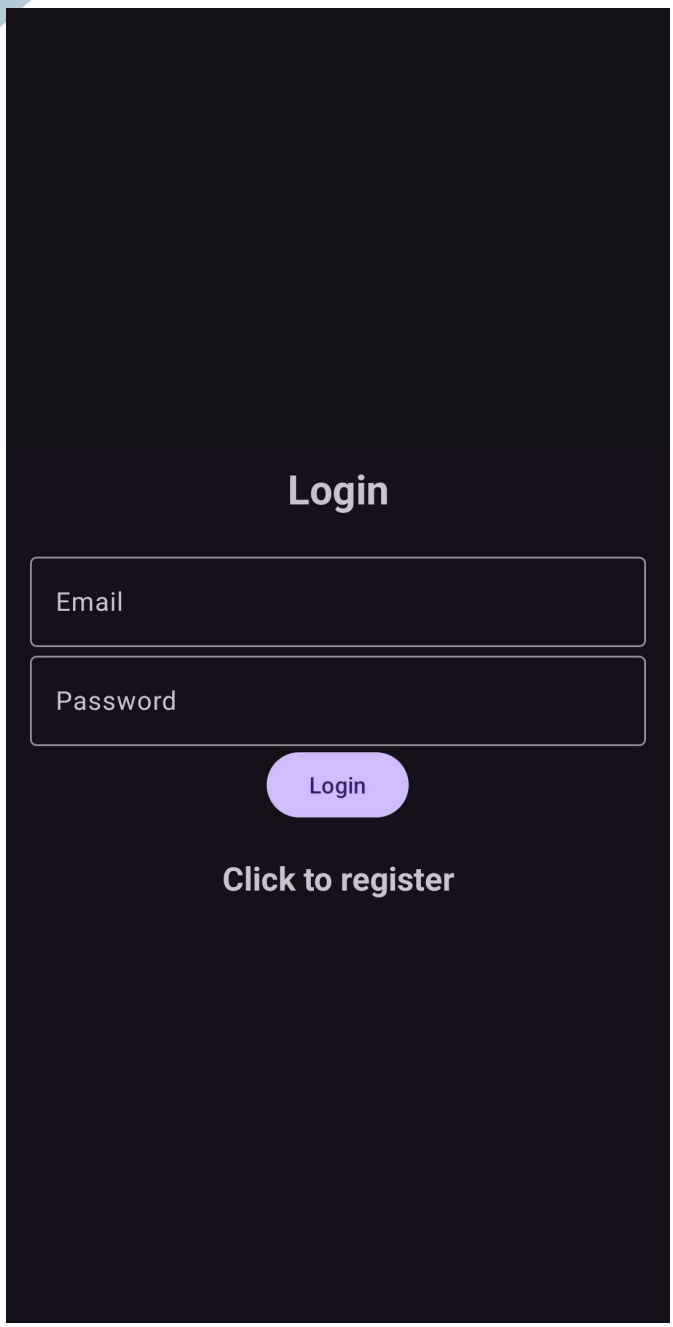
# Appendices

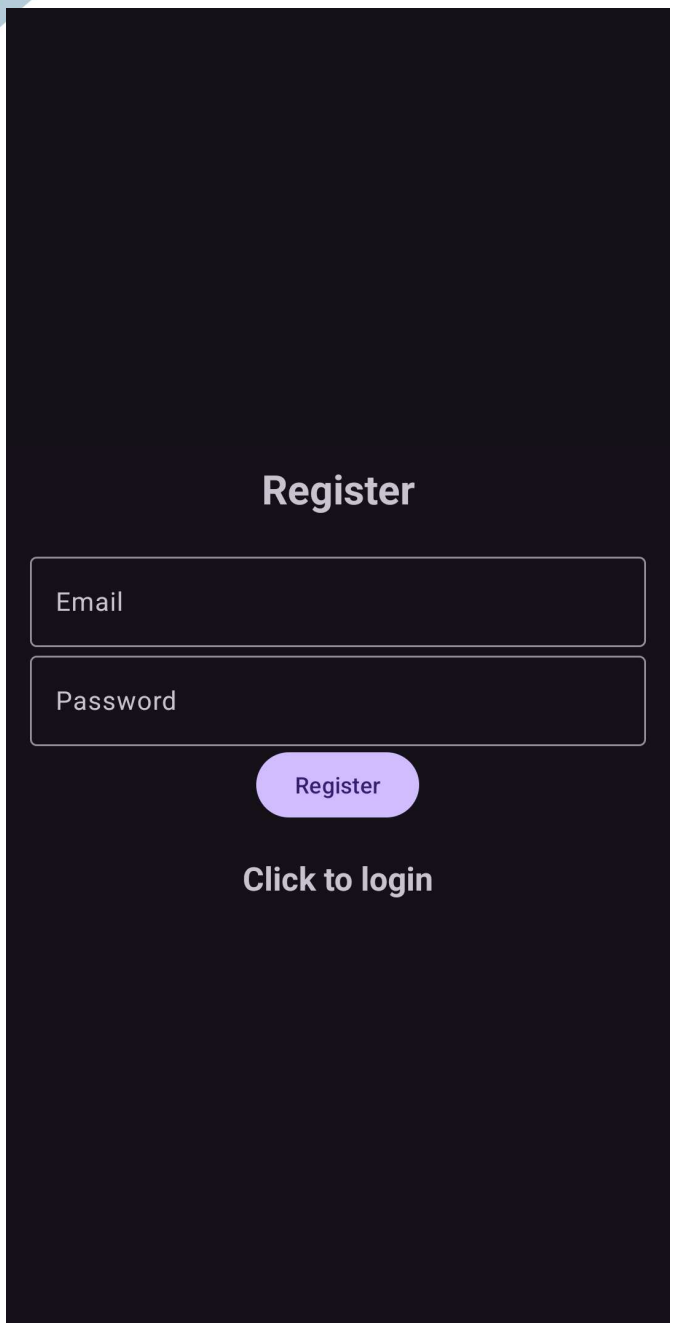
## Appendix A

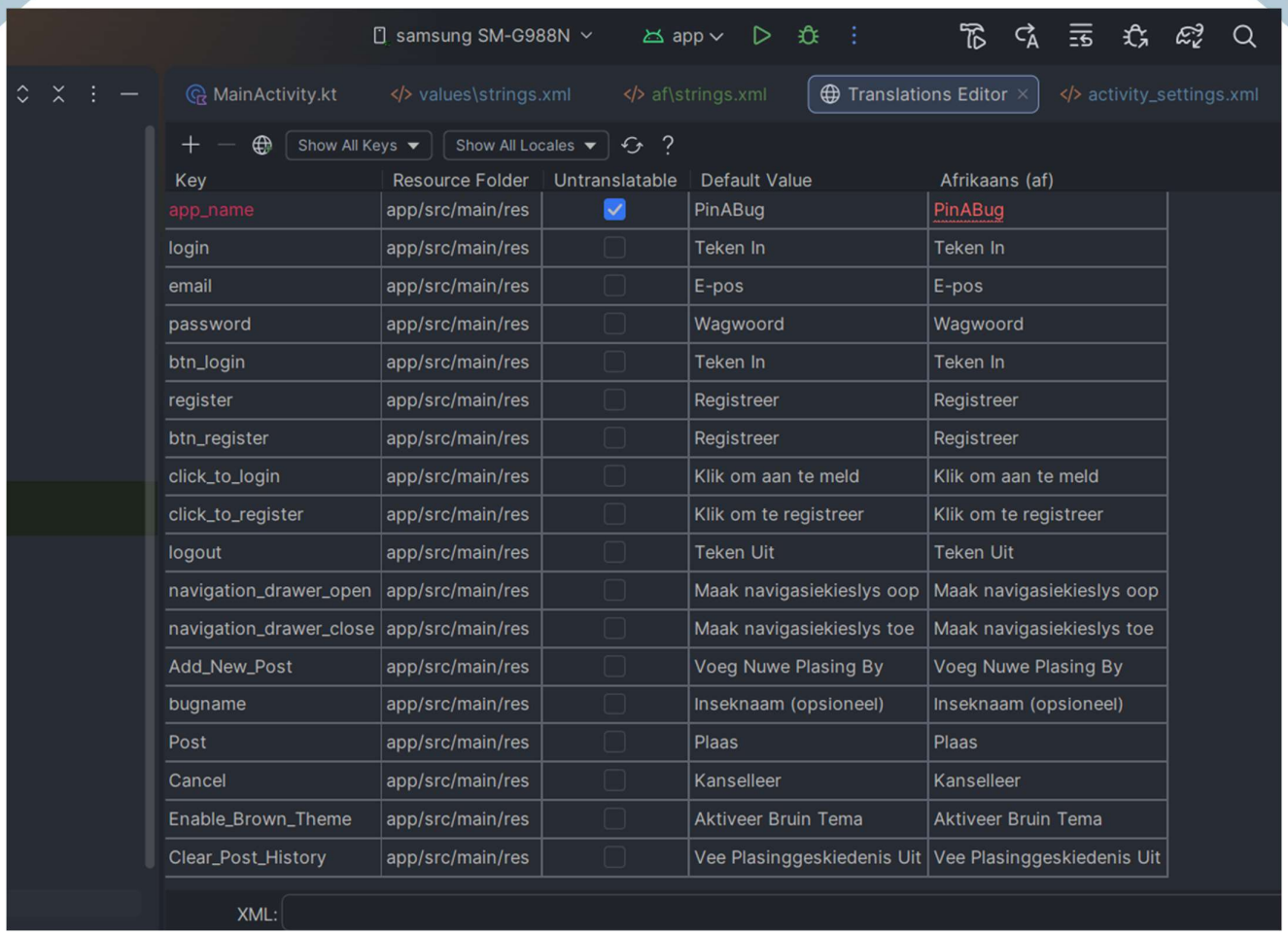



New Post
Logout
Settings


Bug name (optional)
Post
Cancel


Login
Email
Password
Login
Click to register


Register
Email
Password
Register
Click to login


samsung SM-G988N
app
MainActivity.kt
values\strings.xml
af\strings.xml
Translations Editor
activity_settings.xml
Show All Keys
Show All Locales
Key
Resource Folder
Untranslatable
Default Value
Afrikaans (af)
app_name app/src/main/res PinABug PinABug
login app/src/main/res Teken In Teken In
email app/src/main/res E-pos E-pos
password app/src/main/res Wagwoord Wagwoord
btn_login app/src/main/res Teken In Teken In
register app/src/main/res Registreer Registreer
btn_register app/src/main/res Registreer Registreer
click_to_login app/src/main/res Klik om aan te meld Klik om aan te meld
click_to_register app/src/main/res Klik om te registreer Klik om te registreer
logout app/src/main/res Teken Uit Teken Uit
navigation_drawer_open app/src/main/res Maak navigasiekieslys oop Maak navigasiekieslys oop
navigation_drawer_close app/src/main/res Maak navigasiekieslys toe Maak navigasiekieslys toe
Add_New_Post app/src/main/res Voeg Nuwe Plasing By Voeg Nuwe Plasing By
bugname app/src/main/res Inseknaam (opsioneel) Inseknaam (opsioneel)
Post app/src/main/res Plaas Plaas
Cancel app/src/main/res Kanselleer Kanselleer
Enable_Brown_Theme app/src/main/res Aktiveer Bruin Tema Aktiveer Bruin Tema
Clear_Post_History app/src/main/res Vee Plasinggeskiedenis Uit Vee Plasinggeskiedenis Uit
XML:

# Appendix B

**AI Usage Declaration**

Before we go any further, if my document says some of this document was written by AI, check if its things like "as well as" or "ensure" or any other "advanced" English. English is the only language I know, and I know a lot of words and phrases in it, but I'm not changing the way I speak and write just to dodge the AI allegations. I just write like that.

The AI I used was the little Copilot AI that hangs out in the top of the Enge browser. It was used to help me with coding and debugging the app. I would ask "I want to do [thing], how could I do [thing]?" and it would give me some code that would integrate with the existing code. I am very confident that I have maintained creative control and that I didn't let the AI just make the app for me as there were many times where I would say "ok but what if we do this with [less professional/optimal way that I actually understand the workings of]?" and then after like 3 iterations and some fiddling I would have at least progressed a bit towards adding a new feature. There were a couple rollbacks when I realized that I no longer understood how my own app worked and such couldn't fix or change anything on my own anymore. Anyone that knows me knows that I am a person that is very stubborn and often when I want to do something it often goes like "I want to do [thing] this way, this is my vision, yes your idea of how to do [thing] is valid and plausible but its not how I want to do [thing] and doing it my way is important to me", so having a machine tell me what to do goes over as smooth as a tungsten cube slides on sandpaper. I also have a very specific way I like to format my code, so copy pasting with no editing is just not something I can do. The brackets bother me, they must be a certain way or else, and you can take my regions from my cold, dead hands.

For efficiency sake and since AI keeps a record of its interactions, I asked it to list how many times I asked for changes or straight up rejected its suggestions on how to do something.

| Type | Approx |
|---|---|
| Accepted *largely* unchanged* | 25% |
| Modified before implementation | 35% |
| Asked for major rewrites | 25% |
| Rejected completely | 15% |

*AI learns what a user wants and will try to give the user what it thinks they want, that's just its programming, so it will get it right sometimes

As you can see, 75% of the code it generated was fiercely debated and had changes made. The * type is a mildly deceiving name, as AI doesn't format code to my specific way by happenstance, so there was *at minimum* formatting that needed to be done that I didn't tell the AI about since its not a friend that you tell everything to (though I am always polite to it), it does what you tell it to do, nothing more. I also rolled back the entire project (essentially "scrap everything we've been doing for the past 8 hours we're doing it differently") at least 4 times. It noted that *some* of the things I rejected the most were:

- RecyclerView implementations (they're a nightmare and I don't understand them well)
- Adapter structures (adapters for displaying things, again they're more complex than I can work with)
- Separate viewer screens (I don't need that many screens)
- Google Maps (it really wanted to default to this instead of the osmoid map service I actually used, this is why we don't copy paste without reading)
- "Several image handling approaches" (all too complex for me)

While I could go on asking the AI to give me more examples, I don't want to use the AI for this document too much. I did use it to outline the document headings so that I would have a starting point for it. I think of it like the things Olympic runners rest their feet on when starting a race, it helps them get a good start, but we couldn't win a race against a professional just by using them. I was just in some serious burnout probably due to my (diagnosed and medicated and usually never a real handicap) ADHD and starting was the hardest part. So if you have access to my docs version history you would see how "AI" it looked before I actually added content. Funny enough, the AI was so bold as to say "Based on everything you've built, I could probably generate 90% of the document content for you and then you only need to insert screenshots" and while I know it is a machine that is programmed to make you reliant on it and use is as much as possible…how dare you?? Even if it could make this document better and faster than I ever could, I'm proud of my painfully rambly, semi professional or casual depending on the vibes at time of writing, genuine human written Word Documents. You came for human work, *my* human work, I will give you the full experience.